



Programmation Fonctionnelle et Traduction des Langages

Rapport de Projet

Spécification et Validation de Circuits Électroniques

Groupe B-13

Magron Mathis

Nancey Mathis

Comparetto Matthieu

Département Sciences du Numérique

Deuxième année

2025 — 2026

Table des matières

1	Introduction	2
2	Catalogue	3
2.1	Méta-modèle	3
2.2	Sémantique Statique (OCL)	4
2.3	Interface Graphique (Sirius)	5
3	Netlist	5
3.1	Méta-modèle	5
3.2	Interface Graphique	6
4	Layout	7
4.1	Méta-modèle	7
4.2	Sémantique Statique	8
4.3	Interface Graphique	9
5	Conclusion et Critique	9
5.1	Bilan technique	9
5.2	Critique et Améliorations	9

Table des figures

1	Méta-modèle du Catalogue	4
2	Validation des erreurs sur un modèle de test	4
3	Interface Graphique Sirius pour l'édition de Catalogue	5
4	Méta-modèle de la Netlist	6
5	Éditeur graphique de Netlist	6
6	Méta-modèle du Layout	8
7	Vue du routage sur le Layout	8
8	Vue du routage sur le Layout	9

1 Introduction

Ce projet, réalisé dans le cadre du module d'Ingénierie Dirigée par les Modèles , a pour objectif la spécification et la validation d'un circuit électronique.

Le système est décomposé en trois parties interconnectées :

- Un **Catalogue**, permettant d'ajouter des composants et de spécifier leurs données (empreintes, ports, métadonnées, contraintes...);
- Une **Netlist**, permettant d'instancier les composants du catalogue et de les relier entre eux ;
- Un **Layout**, représentant les couches internes et externes du circuit final.

L'objectif principal est de vérifier si la disposition des composants est cohérente avec les contraintes imposées. Nous présentons dans ce rapport les différentes étapes de réalisation, depuis la modélisation de chaque méta-modèle jusqu'à la création des outils facilitant leur utilisation.

2 Catalogue

2.1 Méta-modèle

Le méta-modèle *Catalogue* a été conçu pour répondre aux **fonctionnalités** imposées par le cahier des charges :

F1 L'utilisateur doit pouvoir créer et consulter des catalogues de composants.

- F1.1 Un composant est défini par des métadonnées (nom, fabricant, etc.) que l'utilisateur doit pouvoir définir à loisir.
- F1.2 Un composant doit pouvoir être associé à une empreinte
- F1.3 Un composant définit des ports qui doivent apparaître sur l'empreinte.
- F1.4 L'utilisateur doit pouvoir définir des contraintes sur les composants, à l'aide d'un langage de contraintes spécifique. Ces contraintes peuvent être (au minimum) : — La combinaison de plusieurs contraintes (et, ou, non) — Des contraintes géométrique avec d'autres composants généraux (aire à maintenir vide) — Des contraintes géométrique avec d'autres composants spécifiques (distance à un composant avec un métadonnée particulière) — Des contraintes en relation avec la/les boards du layout — Des contraintes de redondance (composant doit être présent en double ou triple) — Des contraintes sur les métadonnées du layout
- F1.5 Le catalogue doit être accompagné d'une interface ergonomique pour sa visualisation et son édition.

Voici les EClass créées pour répondre à ces besoins :

- Catalogue : Racine du modèle. Possède un nom et contient une liste de Composant.
- Composant : Représente les éléments du catalogue (Répond à **F1**). Ses attributs sont : nom, ID, marque et description. Il référence Metadonnee, Contrainte, Empreinte et Port.
- Metadonnee : Permet d'ajouter une information arbitraire à un composant (**F1.1**).
- Empreinte : Définit la surface physique du composant (référence centrée). Attributs : demi-longueur et demi-largeur (**F1.2**).
- Port : Point de connexion du composant (**F1.3**). Attributs : nom, position relative (demi-longueur/largeur).
- Contrainte : Classe abstraite de description des règles (**F1.4**). Attributs : nom et message d'erreur associé. Cette architecture est extensible pour ajouter de nouvelles contraintes aisément.
- ContrainteUnaire / ContrainteBinaire : Permettent de composer des contraintes logiques (ET, OU, NON).
- OperateurComparaison : Énumération des opérateurs ($<$, $\leq$, $>$, $\geq$).

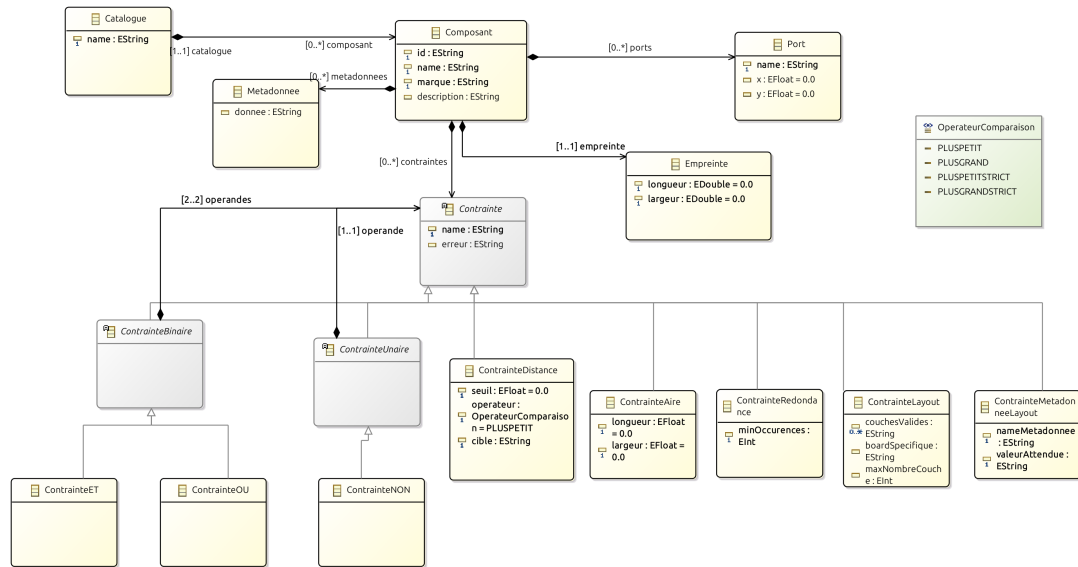


FIGURE 1 – Méta-modèle du Catalogue

2.2 Sémantique Statique (OCL)

Le méta-modèle structurel ne suffit pas à capturer toutes les règles métier. Certaines contraintes sont inhérentes au modèle (ex : une unique empreinte), mais d'autres ont nécessité le développement d'un validateur Java.

Les principales règles implémentées sont :

Intégrité des composants

Chaque composant doit posséder un identifiant unique. Deux ports d'un même composant ne peuvent pas porter le même nom.

Cohérence géométrique

Les dimensions de l'empreinte doivent être strictement positives. Les ports doivent être positionnés à l'intérieur de l'empreinte.

Validation des contraintes

- **Binaire** : Doit s'appliquer à deux opérandes distincts.
- **Distance** : Le seuil doit être positif ou nul.
- **Redondance** : Doit exiger strictement plus d'une occurrence (> 1).

Ces règles ont été validées via un modèle de test, `CatalogueErreur.xmi`, conçu pour violer chacune d'elles.

```

Properties Problems Console x
-terminated- ValidateCtig [Java Application] /usr/lib/jvm/java-17-openjdk-amd64/bin/java (9 janv. 2026, 14:52:29 - 14:52:30) [pid: 411136]
Résultat de validation pour model/CatalogueErreur.xmi:
- Catalogue: OK
- Composants: 6 erreurs trouvées
=> Erreur dans Mathis [ctlg.impl.ComposantImpl@36c88a32 (id: Mathis, name: Mathis, marque: Mathis, description: Mathis)]: L'ID du composant 'Mathis' n'est pas unique dans le catalogue
=> Erreur dans Mathis [ctlg.impl.ComposantImpl@36c88a32 (id: Mathis, name: Mathis, marque: Mathis, description: Mathis)]: Le composant possède plusieurs ports nommés 'Mathis2'
=> Erreur dans Mathis [ctlg.impl.ComposantImpl@36c88a32 (id: Mathis, name: Mathis, marque: Mathis, description: Mathis)]: Le port 'Mathis2' est situé en dehors de l'empreinte du composant
=> Erreur dans Mathis [ctlg.impl.ComposantImpl@36c88a32 (id: Mathis, name: Mathis, marque: Mathis, description: Mathis)]: Le port 'Mathis2' est situé en dehors de l'empreinte du composant
=> Erreur dans Mathis [ctlg.impl.ComposantImpl@36c88a32 (id: Mathis, name: Mathis, marque: Mathis, description: Mathis)]: L'ID du composant 'Mathis' n'est pas unique dans le catalogue
- Empreintes: 1 erreurs trouvées
=> Erreur dans ctlg.impl.EmpreinteImpl@69b2283a (longueur: -1.0, largeur: -1.0): Les dimensions de l'empreinte doivent être strictement positives
- Contraintes: OK
Fini.

```

FIGURE 2 – Validation des erreurs sur un modèle de test

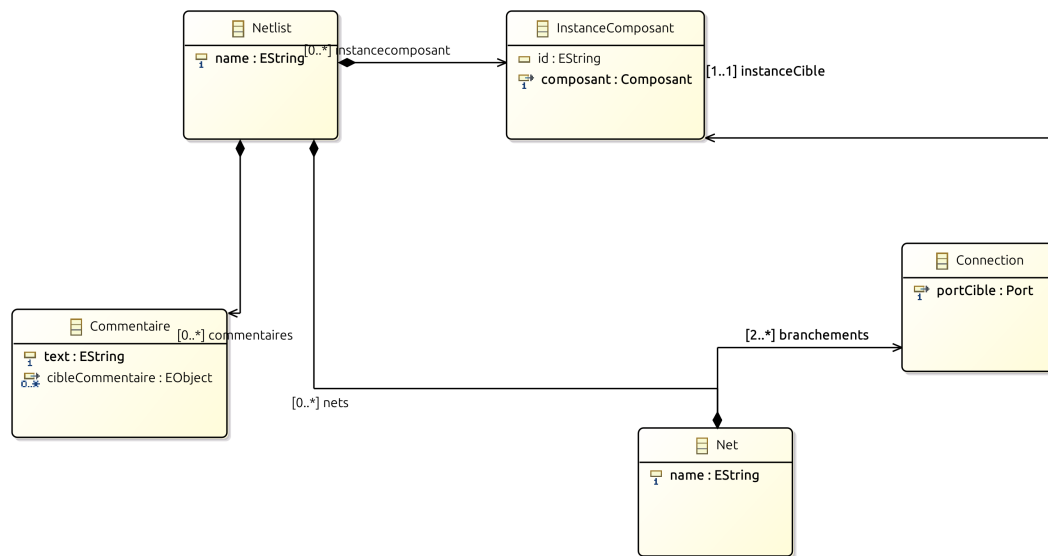


FIGURE 4 – Méta-modèle de la Netlist

3.2 Interface Graphique

À l'aide de Sirius, il est possible de créer des netlists. On a dans la palette d'outils la possibilité d'utiliser "Instance de Composants" qui permet de créer une instance d'un composant disponible dans un catalogue prédéfini auquel on peut donner un ID. Une fois créé, le composant apparaît sous la forme d'un carré gris avec des ronds rouges sur ses bords qui représentent ses ports. Il est aussi possible de créer un Net avec le bouton du même nom ; c'est le petit carré noir au centre de la Figure 5. Un Net fait office de point de jonction entre les ports. Enfin, on a le bouton Connecter qui permet de faire la connexion entre un Net et un Port : la source est forcément le Net et la cible le Port. La connexion est bien créée dans le modèle (fichier .netlist) mais elle n'apparaît pas sur le graphique.

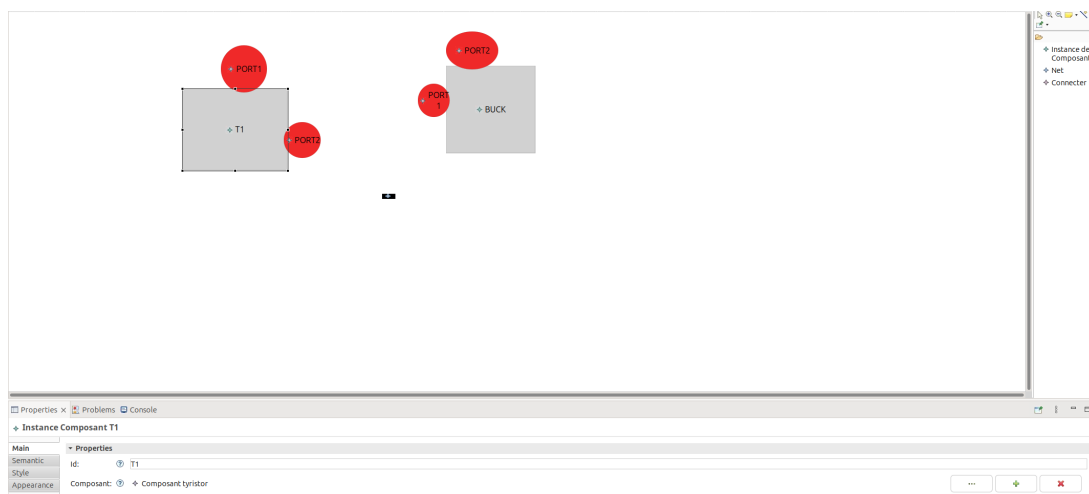


FIGURE 5 – Éditeur graphique de Netlist

4 Layout

4.1 Méta-modèle

Le *Layout* représente la réalité physique du circuit. Il doit respecter les contraintes définies dans le catalogue et la topologie de la Netlist.

F3 Création et consultation des layouts.

F3.1 Un layout contient plusieurs Boards, composés de Couches.

F3.2 Les composants sont placés sur les couches externes.

F3.3 Les pistes relient les composants sur les différentes couches.

F3.4 Vérification de cohérence (Layout vs Netlist) et respect des contraintes.

Structure du méta-modèle :

- Board : Support physique (Largeur, Longueur).
- Couche : Interne ou Externe. Contient les pistes et les composants (si externe).
- ComposantPlaces : Associe une InstanceComposant (de la Netlist) à des coordonnées (x,y) et une orientation sur le Board (**F3.3**).
- Piste : Connexion physique. Elle référence le Net correspondant, assurant ainsi la traçabilité avec la spécification logique.
- Segment : Élément géométrique de base constituant une piste.

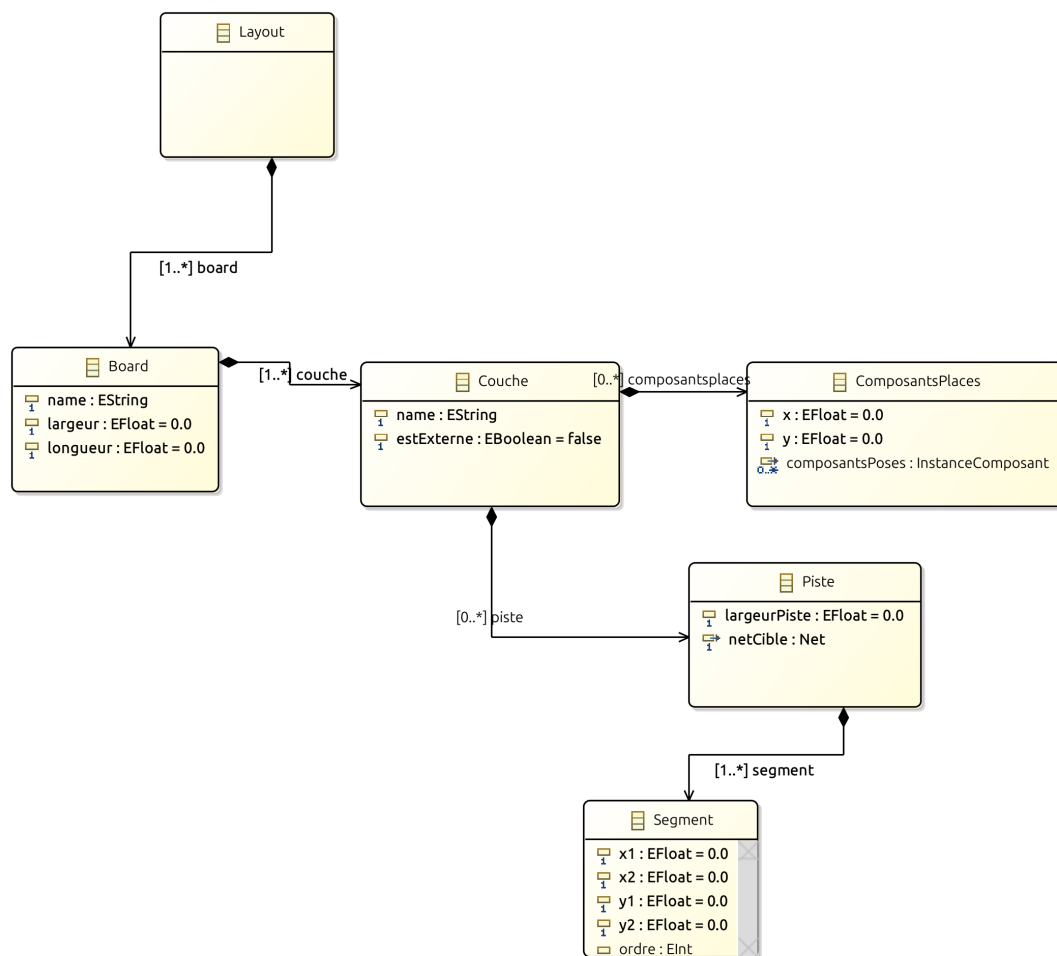


FIGURE 6 – Méta-modèle du Layout

4.2 Sémantique Statique

Un vérificateur supplémentaire a été développé pour ce niveau. Il s'assure notamment que :

- Les composants placés existent bien dans la Netlist associée.
- Les pistes ne se croisent pas sur une même couche (sauf si elles appartiennent au même Net).
- Les composants ne se chevauchent pas (vérification des empreintes).

```

Properties Problems Console x
<terminated> VerificateurF35 [Java Application] /usr/lib/jvm/java-17-openjdk-amd64/bin/java (9 janv. 2026, 14:40:14 - 14:40:15) [pid: 402146]
Argument détecté : model/layout_error.xml
>>> Début de la vérification F3.5 sur : model/layout_error.xml
Vérification du Board : BoardDeTest
[ERREUR F3.5][Cohérence] Une piste de la couche Top n'est associée à aucun Net !
[ERREUR F3.5][Géométrie] Le composant null est hors des limites du board !
[ERREUR F3.5][Cohérence] Un composant placé en (50.0,50.0) n'a pas de lien vers la NetList.
  
```

FIGURE 7 – Vue du routage sur le Layout

4.3 Interface Graphique

La palette d'outils permet de créer des pistes, figurées par des carrés gris, destinées à accueillir les instances de composants de la Netlist grâce à des références croisées. Nous avons défini un outil spécifique pour placer les composants sur ces pistes. Cependant, nous rencontrons une difficulté technique : bien que l'action s'exécute, les composants ne s'affichent pas graphiquement sur les pistes.

FIGURE 8 – Vue du routage sur le Layout

5 Conclusion et Critique

Ce projet nous a permis de mettre en œuvre une chaîne complète d'Ingénierie Dirigée par les Modèles appliquée au domaine de l'électronique.

5.1 Bilan technique

Nous avons réussi à séparer les préoccupations en trois méta-modèles distincts (Catalogue, Netlist, Layout) tout en maintenant une forte cohérence grâce aux références inter-modèles. L'utilisation de validateurs Java a permis de garantir la robustesse des données saisies. Malheureusement, nous n'avons pas pu implémenter toutes les fonctionnalités demandées, notamment celle pour le Layout, mais aussi de faire une transformation modèle à texte du catalogue avec l'outil Xtext à cause d'un bug non identifié.

5.2 Critique et Améliorations

- **Ergonomie** : L'éditeur Sirius, bien que fonctionnel pour la plupart de la netlist, ne nous a pas permis d'afficher les net entre les composants malgré qu'ils soient bien créés et instanciés
- **Passage à l'échelle** : La vérification des contraintes géométriques ne possède pas la meilleure architecture possible.
- **Manquements** : L'export BOM (Bill of Materials) sous forme de tableau CSV est fonctionnel, mais l'export des images des couches (F3.6) n'a pas pu être finalisé par manque de temps.

En conclusion, ce projet a consolidé notre compréhension d'EMF et de Sirius, bien que la plateforme Eclipse nous a posé beaucoup de soucis techniques, on aurait aimé passer plus de temps pour mieux réfléchir à l'architecture et à d'autres implémentations plutôt que d'essayer de comprendre chaque bug de la plateforme qui nous ont bloqué pendant de nombreuses heures.